

Tiered Agent Memory (TAM)

Multi-tier Cognitive Memory Architecture
for AI Agents

minhz99 / tiered-agent-memory

May 4, 2026

Abstract

This document presents **Tiered Agent Memory (TAM)** — a multi-tier cognitive memory architecture for AI Agents. Unlike traditional RAG systems that only store and retrieve data, TAM simulates the cognitive process: the Agent not only knows how to *remember*, but also how to *ignore* (filter noise), *doubt* (when there are contradictions), *generalize* (see the big picture), and *self-learn* from mistakes. The architecture consists of **4 storage tiers** (Working, Active, Latent, Archive), **3 dynamics mechanisms** (Activation, Decay, Reinforcement), and **7 horizontal control planes** (Query Understanding, Competition, Abstraction, Reasoning, State, Scaling, Evolution). The paper also presents a Python implementation illustrating the full end-to-end 12-step pipeline.

Contents

1	Document Objectives	3
2	Overall Model: 4 Tiers + 3 Mechanisms + 7 Control Planes	4
3	Core Design Principles	5
4	Tier 1: Working Memory — The Current Thinking Space	7
5	Tier 2: Active Memory — Stable Knowledge, Default Access	8
6	Tier 3: Latent Memory — Dormant Memories Requiring Activation Cues	9
7	Additional Layer: Abstract Memory — The Generalization Layer to Reduce Tokens and Increase Understanding	10

8	Additional Layer: Reasoning Memory and ReasoningBank	11
9	Tier 4: Archive — Cold Storage, Serving Deep Retrieval	12
10	State Specification of a Memory	13
11	End-to-End Data Flow Specification	13
12	Upgraded Operational Mechanisms	15
13	Rules for Creating, Updating, Promoting, and Self-Evolving	17
14	Style Memory and Context Override	19
15	Cost, Performance, and Scalability Optimization	19
16	Common Design Errors	20
17	Evaluation Criteria for a Practical Memory System	21
18	Core Conclusion	22
A	Overall Architecture Diagram	23
B	Summary Comparison Tables	23
C	Core Algorithms	24
D	Python Source Code Implementation	25
E	References	26

1 Document Objectives

This document describes a memory architecture that can be used to design, evaluate, and operate Agents in practice. The core of the architecture remains the 4-tier storage model, but the goal of the document is not just to provide a “place to remember”, but to clarify the **memory coordination system**: what is remembered, what is ignored, what should only be activated when cued, what must be doubted, what should be generalized, and what must be converted into reusable reasoning strategies.

If memory is viewed merely as a data warehouse, the system will quickly encounter three problems: retrieval noise, Working Memory bloating, and the generation of “split personality” responses when old, new, correct, and incorrect information all crowd into the context. Therefore, a good memory system must simultaneously satisfy four objectives:

- **Contextually correct:** Bringing the right knowledge or the right strategy into the right response turn.
- **Economical:** Keeping Working Memory small enough not to waste tokens and latency.
- **Adaptive:** Allowing memory to change over time, according to new evidence and execution results.
- **Controllable:** Explaining why which memory is selected, which memory is inhibited, which memory is demoted, which memory is marked obsolete, and which memory is elevated into a general strategy.

The current architecture is on the right track thanks to stratification, Latent Memory, trigger-based recall, competition, abstract memory, and intent-based gating. However, to truly reach the production-ready level and become closer to human memory, the system needs four core upgrades:

- **Reasoning Memory:** Storing generalizable problem-solving strategies, rather than just facts or events.
- **Contrastive Signals:** Learning not only from successes but also from failure trajectories and dead ends.
- **MaTTS (Memory-aware Test-Time Scaling):** When the budget is high, the system not only retrieves deeper but also expands multi-branch reasoning guided by memory.
- **Self-Evolving Background Worker:** A background worker that not only cleans up memory but also curates logs into higher-quality memories and injects them back into Active Memory.

In short: the current system already “knows how to remember”, but to truly become more powerful it must also learn how to “reason again, learn from mistakes, scale at the right time, and self-evolve”.

2 Overall Model: 4 Tiers + 3 Mechanisms + 7 Control Planes

The architecture should be understood as three overlapping layers.

The first layer is the **four storage tiers**, answering the question *where does memory reside and what is its availability level*. These four tiers are:

- **Working Memory (WM):** The active context layer in the current request.
- **Active Memory:** The stable, fast-access knowledge layer, which is the default retrieval source.
- **Latent Memory:** The infrequently used memory layer, only activated when there is a strong enough cue.
- **Archive:** The cold storage layer, serving tracing, offline analysis, and rare recovery.

The second layer is the **three dynamics mechanisms**, answering the question *how does memory surface, fade, or become strengthened*. These three mechanisms are:

- **Activation:** The mechanism that makes a memory or a strategy a candidate for the current reasoning turn.
- **Decay:** The mechanism that reduces the probability of a memory being selected over time, by memory type, and by the level of disuse.
- **Reinforcement:** The mechanism that consolidates memories that have been used and proven useful.

The third layer is the **seven horizontal control planes** – deciding whether the architecture is truly production-ready and close to human memory. These seven planes are:

- **Query Understanding Plane:** Intent classification, query rewriting, cue expansion.
- **Competition Plane:** Competition, inhibition, dedup, and selection among competing memories.
- **Abstraction Plane:** Creating abstract memories and extracting general principles.
- **Reasoning Plane:** Managing Reasoning Memory and ReasoningBank as a repository of reusable strategies.
- **State Plane:** Managing conflicts, validity time, currency, and forgetting strategies by memory type.
- **Scaling Plane:** Managing MaTTS, i.e., memory-guided test-time scaling.

- **Evolution Plane:** Managing contrastive signals, reflection, and self-curation in the background worker.

The key point is: **the 4 tiers are just the storage topology**. For the system to be more “human-like” and practical to use, control planes are needed so that it not only knows how to remember but also knows how to choose strategies, learn from mistakes, expand reasoning when needed, and rewrite memories at a higher quality level.

3 Core Design Principles

Principle 1: Memory is an activation system, not a storage warehouse

A larger warehouse does not mean a better memory system. The quality of memory must be measured by the ability to activate the right knowledge or strategy at the right time, not by the number of records stored.

Principle 2: Queries must be re-interpreted before retrieval

Humans rarely access memory using the exact words just heard. The brain often reinterprets the question into adjacent concepts. Therefore, a good Agent should not retrieve directly from the original query but should have a *query rewriting/query expansion* step to expand from the surface text to concepts that are truly useful for recall.

Principle 3: Memories must compete, not just be ranked

Taking the top- K by score is not enough. Related memories must **compete** to be activated, and a stronger memory must be able to **inhibit** similar or redundant memories. Without competition/inhibition, the system will often bring into WM many memories that are identical or almost correct but wrong in context.

Principle 4: The system must have atomic, abstract, and reasoning memory

Atomic memory is great for storing a specific detail. Abstract memory is great for grasping the “big picture”. But for the Agent to truly learn how to solve new tasks, **Reasoning Memory** is needed – that is, the layer that stores problem-solving strategies that can be reused across different situations.

Principle 5: The system must learn from both successes and failures

If reinforcement only rewards what was done correctly, the system misses half the important lessons. Dead-end trajectories, trial-and-error branches, and ineffective strategies are highly valuable **contrastive signals** to extract sharper principles.

Principle 6: Time, conflict, and currency are first-class properties

A memory not only needs to be correct in content; it also needs to be correct regarding *when*. Without temporal awareness and conflict resolution, the system easily retains states or strategies that have expired.

Principle 7: Intent is just as important as domain

Domain gating is necessary, but not sufficient. A personal memory about the gym might share a broad domain with a question about optimizing a diet, but it is entirely unnecessary if the current intent is analyzing an algorithm. Therefore, memory usage must be filtered by **intent**, not just by domain.

Principle 8: Not all types of memory forget the same way

Episodic memory should be forgotten faster than semantic memory; reasoning memory should only weaken when that strategy is counter-exemplified or no longer effective; system memory should almost never undergo decay by the same mechanism as episodic memory.

Principle 9: High budget must scale reasoning, not just retrieval

Old-style attention budgets answer the question “how many memory tiers to open”. MaTTS goes further: when the problem is hard, a high budget must activate multi-branch reasoning, sequential or parallel, guided by memory to avoid blind exploration.

Principle 10: Self-evolution should be the background worker’s responsibility

The online path must be fast, neat, and safe. Heavy work like reading all daily logs, comparing successes with failures, synthesizing general principles, and rewriting higher-quality memories should be delegated to a background worker.

4 Tier 1: Working Memory — The Current Thinking Space

System Role

Working Memory is the context portion that the model sees in the current response turn. With LLMs, this is where the highest operational power lies: what is not brought into the WM is almost entirely unable to directly affect the response.

WM is not a place to “store” but a place to **compose**. It should be built from five main sources:

- The current request.
- The short-term history of the conversation.
- The system instructions currently in effect.
- A small set of memories that have gone through query rewrite, retrieval, competition, conflict filtering, and compression.
- Control parameters such as recall confidence, strategy hints from ReasoningBank, and the current scaling mode of MaTTS.

Minimum Specification

A good WM should follow these rules:

- **Budget cap:** The memory part only occupies a small fraction of the total context.
- **Small top- K :** Only keep the very best candidates.
- **Competition before injection:** Do not allow many similar memories into WM at the same time.
- **Atomic + abstract + reasoning mix:** When appropriate, prioritize a small mix of details, generalizations, and strategies instead of many disjointed pieces.
- **Intent-aware gating:** If the current intent is technical, avoid pulling irrelevant personal memories.
- **Meta-memory:** If recall confidence is low, WM must reflect that uncertainty rather than presenting the memory as an absolute truth.
- **Budget-aware reasoning:** If MaTTS is in a high scaling mode, WM needs to leave room for reasoning branches and guiding strategies.

Core Trade-off

The central trade-off of WM is not only between “a lot of information” and “few tokens”, but also between **detail**, **generalization**, and **strategy**. Introducing too many atomic memories will make WM heavy and confusing; introducing abstract memory too early can lose important details; introducing overly general strategies can make the response “purely theoretical”. Therefore, WM should always be viewed as a controlled mix of detailed representations, generalized representations, and strategy hints.

5 Tier 2: Active Memory — Stable Knowledge, Default Access

System Role

Active Memory contains knowledge with a high probability of reuse: semantic memory, style profile, system memory, and verified Reasoning Memories. These are what the Agent should “think of by default” first.

Criteria for a Memory to be in Active

A memory should only reside in Active if it satisfies most of the following conditions:

- Has relative stability.
- Useful for many future turns.
- Has clearly defined domain and intent scopes.
- Has high reliability.
- Can be represented compactly.
- Is not just a short-term state that is about to expire.
- If it is a strategy, there must be evidence that it has helped solve a class of problems, not just a single case.

Retrieval Mechanism

Retrieval from Active should be the default, but not *always-include*. Active only provides candidates. Afterwards, the candidate still has to pass the intent filter, competition, conflict filter, and budget filter before being allowed into WM.

Within Active, at least four families of memories should be separated:

- **Semantic/personal memory:** User preferences, roles, constraints.
- **System memory:** Policies, environment, tool capabilities.
- **Style memory:** Preferences for length, tone, response structure.
- **Reasoning memory:** Problem-solving strategies that have been generalized and verified.

Core Trade-off

The biggest risk of Active is not only **over-personalization**, but also **over-trusting old strategies**. A reasoning strategy that was once correct in many cases can still become wrong when the context changes. Therefore, reasoning memory in Active must be accompanied by a clear validity scope and validation state.

6 Tier 3: Latent Memory — Dormant Memories Requiring Activation Cues

System Role

Latent Memory stores episodic memory, transient strategies, reasoning traces, and infrequently used details that still have potential value: old incidents, short-term habits, past projects, symptoms previously mentioned, strategies tried, and dead-end trajectories.

Activation Principles

Latent should not be accessed by default in every turn. It should only activate when there is a strong enough cue after the query rewriting step. These cues can come from:

- **Keyword cue:** Exact or near-exact match of important keywords.
- **Semantic cue:** The new query is semantically close to an old experience.
- **Explicit cue:** The user explicitly asks to recall something.
- **Risk cue:** The current problem has a high risk if an old memory is missed.
- **Strategy cue:** The current problem is similar to a family of problems where the system previously succeeded or failed.

Two-pass retrieval as default specification

A reasonable process is:

1. Pass 1: The rewritten query is used to fetch candidates from Active and ReasoningBank.
2. Pass 2: If the trigger is strong enough, the expanded query continues to be used to fetch candidates from Latent.

The new point to emphasize is: **the query for Latent should not be the original query**. It should be the re-interpreted query to increase recall. This is a huge difference between “having latent memory” and “using latent memory effectively”.

Core Trade-off

Latent allows the system to recall rare things, but it is also the place most prone to false activation. Therefore, Latent always needs to be accompanied by competition, inhibition, recall confidence, and contrastive filtering. The system not only needs to ask “which memories are related”, but also “which memory is worth trying again” and “which failed memory should be used as a warning instead of advice”.

7 Additional Layer: Abstract Memory — The Generalization Layer to Reduce Tokens and Increase Understanding

Why it is needed

An architecture with only atomic memory will be very clean in terms of retrieval, but lacks a layer of generalization. When the user has many small memories centered around a topic, the system needs a high-level representation to capture the “big picture” without loading too many details into WM.

For example, atomic memories like “likes gym”, “once asked about diet”, “often asks about calories” can be merged into an abstract memory: “User cares about health and fitness.”

Architectural Nature

Abstract Memory doesn’t necessarily have to be an independent “fifth storage tier” separate from the original 4 tiers. The more correct way is to view it as a **horizontal synthesis layer** running on top of Active and Latent. It is created by merging related memories and is used primarily for:

- Token-saving context injection.
- Maintaining consistent thematic continuity.
- Increasing reasoning ability at a generalized level.
- Acting as a representative in competition when many atomic memories on the same topic compete.
- Serving as a premise to distill Reasoning Memory from many individual cases.

Core Trade-off

Abstract memory helps reduce tokens and increase coherence, but if abstraction happens too early, the system loses important details. Thus, abstract memory should be viewed as a **supplementary** layer, not a complete replacement for atomic memory.

8 Additional Layer: Reasoning Memory and ReasoningBank

Why it is needed

The existing memory tiers mainly answer the question “What does the Agent know about the user, the system, and the past?”. But for the Agent to carry skills from an old task to a new task, an additional layer is needed to answer a different question: “**What problem-solving strategies has the Agent learned?**”

Reasoning Memory does not store “knowledge” in the sense of a single fact, but stores **Generalizable Reasoning Strategies**. For example, instead of just remembering that a specific bug was fixed in which commit, it stores the principle: “When encountering error X in module Y, check the config file Z first because there are often asynchronous errors.” This type of memory has a much higher ability to generalize than episodic memory.

Where does it sit in the architecture

Operationally, Reasoning Memory sits at the intersection of Active and Abstract:

- It belongs to **Active** because it must be accessed quickly when encountering similar problems.
- It belongs to **Abstract** because its nature is the generalized result of many experiences.
- The raw trajectories that generated that strategy might reside in **Latent** or **Archive**.

Therefore, a reasonable implementation is to have a **ReasoningBank** as a sub-bank of Active/Abstract, which contains validated strategies.

Core Trade-off

Reasoning Memory is very powerful because it helps the Agent not have to “reinvent how to think” for each new task. But it is also dangerous if the strategies are distilled from too few examples or from an overly narrow context. Therefore, reasoning memory must be attached to an **applicability scope**, **validation status**, and **contrastive evidence** rather than just a beautiful summary sentence.

9 Tier 4: Archive — Cold Storage, Serving Deep Retrieval

System Role

Archive holds full history, snapshots, raw traces, successful and failed trajectories, and data for auditing or offline analysis. This is where the problem of “storing to survive” is solved, not a place for permanent reasoning.

Operational Specification

The Archive should have:

- Minimal indexing by time, user, domain, event type, and trajectory type.
- Snapshot summaries for fast re-hydration.
- Clear retention policies.
- Ability to selectively extract up to Latent, Active, or ReasoningBank when needed.
- Clear links so the worker can trace back successful and failed raw trajectories for the self-curate step.

Core Trade-off

An Archive without indexing or snapshot summaries is just a log dump. The value of Archive does not lie in “keeping a lot”, but in **being able to recover the right things when needed at an acceptable cost**, and recovering both the right and the wrong to serve contrastive learning.

10 State Specification of a Memory

For the architecture to be truly deployable, each memory needs a minimum state rich enough to support retrieval, conflict resolution, temporal awareness, reasoning transfer, and explainability. A minimal record should carry the following groups of properties:

- **Identity:** ID, storage tier, and memory type.
- **Content:** Content summary, abstraction group if any, and if it's a reasoning memory, the core strategy and application scope.
- **Context:** Domain tags, intent tags, module tags, or task family if needed.
- **Source and Reliability:** Origin of the memory, confidence, recall confidence, and verification state.
- **Time:** Validity start time, validity end time, currency flag, creation time, last verified time, and last used time.
- **Behavior:** Usage count, reinforcement score, decay profile.
- **Contrastive Evidence:** References to success cases, failure cases, causes of failure, and reflection summaries.
- **Conflict:** Conflict status, which memory it replaces or is replaced by.

Not every memory needs to have all fields fully populated, but without these groups of properties, the system will have a hard time solving the problem of “which memory is correct, still valid, reliable, transferable, and should be chosen right now”.

11 End-to-End Data Flow Specification

A production-ready processing turn should be described as a twelve-step pipeline.

Step 1: Intent, domain, and complexity analysis

The system determines what kind of question the user is asking: technical, personal, retrospective, decision-making, or simple. This is the first step that decides which memories are allowed to participate and whether the task requires test-time scaling.

Step 2: MaTTS budget setup

Instead of just deciding “how many memory tiers to open”, the system determines the scale of reasoning at runtime. If the query is simple, the system takes the fast path: minimal retrieval, no multi-branch expansion. If the query is difficult, ambiguous, or high risk, the system turns on **MaTTS** to allow deeper sequential or parallel reasoning.

Step 3: Query rewriting / query expansion

The user’s original query is re-interpreted into concepts better suited for memory retrieval. This expands the surface language into concepts useful for retrieval and for selecting strategies in the ReasoningBank.

Step 4: Intent-aware gating

Before searching for memories, the system eliminates memory families incompatible with the intent. For example, if the intent is algorithm explanation, a personal memory about the gym shouldn’t compete even if they share a broad “self-improvement” domain.

Step 5: Background retrieval from Active and ReasoningBank

The rewritten query is used to search Active via vector search, metadata filtering, and domain filtering. Simultaneously, the system queries ReasoningBank for strategies that succeeded in similar problem families.

Step 6: Latent cue detection

The system checks if there are keyword cues, semantic cues, explicit cues, risk cues, or strategy cues strong enough to open Latent.

Step 7: Expanded retrieval from Latent, Abstract Memory, and raw traces

If cues exceed the threshold, the system uses query expansion to search Latent. Also, if multiple atomic memories of the same topic appear, the system might retrieve abstract memories representing that group. For very hard queries, it might also fetch trace summaries from Archive to provide context for reflection or branch guidance.

Step 8: Competition, inhibition, and conflict filtering

Candidates are not just ranked by score, but **compete**. Stronger memories can inhibit near-duplicates or less useful ones. Conflicting memories must be flagged or de-prioritized before entering WM.

Step 9: Working Memory construction

WM is built from a small mix of atomic memories, abstract memories, reasoning memories, and control parameters like style overrides or recall confidence. If recall confidence is

low, the system should prepare phrasing reflecting uncertainty rather than over-asserting.

Step 10: MaTTS activation and multi-branch reasoning

If the budget is high enough, the system does not stop at a single reasoning pass but activates **Memory-aware Test-Time Scaling**. Here, the Agent might run reasoning branches sequentially or in parallel, similar to Tree of Thoughts, but not blindly: ReasoningBank provides strategy hints to guide exploration more efficiently.

The correct relationship here is:

- **Memory guides Scale:** Old strategies keep new branches from wandering.
- **Scale feeds back to Memory:** New branches create various solution paths, both right and wrong, providing data for contrastive learning.

Step 11: Answer selection, recording successes and failures

After reasoning branches finish, the system selects the final answer based on intrinsic quality, consistency, and confidence. More importantly, it does not only record the winning branch. Losing branches, dead ends, and causes of failure are also kept as **contrastive signals**.

Step 12: Update, reflection, and self-evolution schedule

After generation, the system updates reinforcement for useful memories, applies decay by memory type, marks conflicts as outdated, updates validity time on state changes, and creates a **reflection summary** like: “Strategy A succeeded, Strategy B failed because cause C.” This reflection serves short-term updates and is queued for the background worker to distill later.

12 Upgraded Operational Mechanisms

Activation + Competition

Activation should not just be adding similarity, importance, recency, and confidence. It needs a **competition** component. A memory is selected not only because it is strong, but because it beats near-duplicates or direct competitors.

Competition prevents two common errors:

- Many similar memories crowding into WM causing noise.
- Almost-correct but out-of-context memories winning due to high similarity.

Decay + Forgetting by Memory Type

Decay must depend on the type of memory. A reasonable profile is:

- **Episodic memory:** Fast decay.
- **Semantic memory:** Slow decay.
- **Reasoning memory:** Slow decay but sensitive to counter-examples and failure signals.
- **Style/profile memory:** Medium decay and depends on recent confirmations.
- **Skill/system memory:** Almost no decay by the same mechanism as episodic.

Reinforcement + Contrastive Signals

Reinforcement shouldn't just increase scores for what was "done right". The system must learn from both successful and failed experiences. Dead-end trajectories shouldn't be completely discarded; they should be kept as **contrastive signals** helping the system distinguish between "strategies to try again" and "strategies to avoid".

A good reflection doesn't just say "Approach A is effective", but goes further: "A is more effective than B because B fails when assumption C is false." This contrast is what creates sharp reasoning memory.

Reasoning Memory Formation

Reasoning Memory is formed when the system distills multiple traces into a generalizable, transferable strategy. Crucially, it must not be generated from a single case if that case is not representative enough. A strategy should only be pushed to ReasoningBank when:

- It has appeared or been confirmed in multiple situations.
- Its applicability scope is clear.
- There is contrastive evidence showing why it is better than competing strategies.

Conflict Resolution

This is a mandatory mechanism to avoid "split personalities". When a new memory conflicts with an old one, the system must have a clear policy:

- Reduce the confidence of the old memory.
- Mark the old memory as outdated or superseded.

- Only keep the old memory in Latent/Archive if it still has historical value.
- Do not allow both into WM as two current truths.
- If the conflict is at the strategy level, explicitly note which strategy is counter-exemplified by the new case.

Temporal Awareness

Recency and TTL are not enough. The system needs to distinguish between **current memory** and **formerly true memory**. For reasoning memory, temporal awareness must also answer: is this strategy still relevant, or only true for an older system version, configuration, or environment.

Meta-memory

Meta-memory is the ability to know “I don’t remember for sure” or “I have an old strategy, but I’m not sure if it still applies”. This step brings the system closer to human memory. If recall confidence is low, the response should use frames like: “I’m not entirely sure, but it seems that in the past...” instead of presenting the memory as an absolute fact.

MaTTS: Memory-aware Test-Time Scaling

MaTTS is a direct upgrade to Attention Budget. Instead of just deciding whether to turn latent retrieval on or off, MaTTS decides the **scaling level of the reasoning process**. When the task is hard, the system can:

- Increase the number of reasoning branches.
- Run sequentially or in parallel.
- Use ReasoningBank to orient the branches.
- Compare trajectories to generate new contrastive evidence.

Thus, MaTTS is not just a token-saving strategy. It is the mechanism that allows memory and reasoning to act symbiotically.

13 Rules for Creating, Updating, Promoting, and Self-Evolving

Creation

Not every utterance should be saved. A good write path evaluates:

- The clarity of the utterance.
- The expected durability.
- The value for future turns.
- The domain and intent scope.
- The potential for conflict with existing states.
- The memory type to be assigned if saved.
- Whether this is just an isolated experience or has enough substance to be distilled into reasoning memory.

Reflection from Successes and Failures

After hard tasks, the system should generate short reflections answering three questions:

- Which strategy succeeded.
- Which strategy failed.
- Did the failure stem from false assumptions, missing information, or misapplication of context.

This reflection bridges raw trajectories and Reasoning Memory.

Promotion

A memory should be elevated from Latent to Active when it is not only recalled multiple times but also demonstrates stability, few conflicts, and high reuse value. For reasoning memory, promotion demands more: the strategy must prove its ability to generalize across multiple tasks, not just a single case.

Demotion

A memory in Active should be demoted to Latent when it is no longer frequently used, no longer a default priority, or has lost its currency but remains worth keeping as historical context. For reasoning memory, demotion occurs when a once-useful strategy meets many new counter-examples.

Archiving

Very old memories with low recall and almost no value for daily reasoning should be moved to Archive. But archiving doesn't mean deleting. It is moving from "hot reasoning" to "cold storage", while still retaining raw materials so the worker can re-synthesize when needed.

Self-Evolving Background Worker

This is the part that helps the architecture truly “evolve”. The background worker should do more than garbage collection. Periodically, it must:

- Gather all logs and trajectories of daily tasks.
- Separate success and failure cases.
- Call an LLM acting as a **Synthesizer** to analyze why the Agent failed at one task but succeeded at another today.
- Distill general principles into higher-quality memories.
- Inject the best memories back into Active Memory or ReasoningBank after passing a validation gate.

This is when the system transitions from maintenance to **self-curation**. That is, not only keeping memory cleaner but also making memory smarter over time.

14 Style Memory and Context Override

The style profile is a very useful part of Active Memory, but style should not be a hard rule. It must be adjusted by the current context. A reasonable way to think about it is:

$$\text{final_style} = 0.7 \text{user_style} + 0.3 \text{context_style}$$

The specific coefficients are just illustrative. The important point is: the user’s preferred style should be respected, but when the current task is technical analysis, mathematical proof, or production bug handling, the context style must have the authority to pull the response toward rigor.

15 Cost, Performance, and Scalability Optimization

The most important optimization levers of this upgraded system are:

- **Adaptive MaTTS:** Only scale multi-branch reasoning when truly necessary.
- **Conditional query expansion:** Only expand deeply when there are signs of latent recall or high risk of omission.
- **Competition before injection:** Reduce the number of topically redundant memories entering WM.
- **Abstract memory:** Reduce tokens while preserving the “big picture”.

- **Reasoning memory:** Reuse strategies instead of rediscovering from scratch on every task.
- **Contrastive logging:** Turn failures into training data rather than lost costs.
- **Background self-curation:** Move heavy synthesis work to the worker, keeping the online path neat and fast.

A production-ready system isn't one that always "thinks the deepest", but one that knows when to think deeply, when to use old strategies, when to expand multi-branches, and when to let the background worker synthesize the lessons.

16 Common Design Errors

Error 1: Only having top- K without competition

This allows many similar memories to enter WM, or almost-correct but out-of-context memories aren't strongly inhibited.

Error 2: Retrieving directly from the original query

Latent Memory exists but recall is very weak because the system does not reinterpret the question into necessary concepts.

Error 3: Only having atomic memory without abstract memory

The system either has to load many small fragments into WM or loses the ability to grasp the big picture of the user and the situation.

Error 4: No Reasoning Memory

The Agent only knows facts and old experiences but cannot truly transfer "problem-solving strategies" from one task to another.

Error 5: Only learning from successes and ignoring failures

The system continuously rewards what was right but never clearly understands why other reasoning paths were wrong, leading to shallow strategy learning and repeated dead ends.

Error 6: No conflict resolution

Old and new memories coexist as two current truths, leading to contradictory responses.

Error 7: No temporal awareness

The system fails to distinguish between “is currently like this” and “used to be like this”, or “strategy that used to be useful” and “strategy that is still relevant”.

Error 8: Budget only controls retrieval without scaling reasoning

High-budget mode just spends tokens finding more memories, but the Agent still thinks along a single path and misses the benefits of test-time scaling.

Error 9: Background worker only cleans up without self-curation

The system might get cleaner over time, but not truly smarter over time.

Error 10: No meta-memory

The Agent appears either “certain” or “ignorant” without being able to express a reasonable state of uncertainty.

17 Evaluation Criteria for a Practical Memory System

To evaluate this system in production, the following questions should be asked:

- Does query rewriting actually increase latent recall without overly inflating false recall?
- Does competition reduce redundancy in WM?
- Does abstract memory help reduce tokens while maintaining the major topic?
- Does reasoning memory help transfer strategies from old tasks to new ones?
- Do contrastive signals truly make reflections sharper?
- Does conflict resolution prevent contradictory memories from both being considered current?
- Does temporal awareness correctly distinguish between current and past states?
- Does MaTTS improve quality on hard queries without causing costs to explode on simple ones?
- Does the background worker create truly useful higher-quality memories or just generate more noise?

- Is recall confidence well-calibrated enough so that responses reflect appropriate uncertainty?

A good system not only answers well but must allow the designer to see why it chose to remember this, ignore that, doubt the other, learn from which mistake, and distill what new principle.

18 Core Conclusion

The current 4-tier architecture is on the right track because it already knows how to **stratify**, has **Latent Memory**, has **trigger-based recall**, has **competition**, and has **abstract memory**. That is the “knowing how to remember” and “knowing how to choose” part of the system.

But to be truly close to human memory and robust enough for production, the architecture needs to implement the next four core upgrades:

- **Reasoning Memory / ReasoningBank:** So the Agent knows not just facts, but strategies.
- **Contrastive Signals:** So the Agent learns from both successes and failures.
- **MaTTS:** So a high budget doesn’t just open more memories but scales guided multi-branch reasoning.
- **Self-Evolving Background Worker:** So the system is not only cleaned up but also self-curates and self-evolves.

If the entire specification had to be summarized in one short sentence, it would be: **a good memory system not only knows how to remember, but also knows how to extract strategies, learn from mistakes, and rewrite its own memory**. These three capabilities are what take the 4-tier architecture from the level of “conceptually correct” to the level of “sharp enough to run for real and alive enough to self-evolve”.

A Overall Architecture Diagram

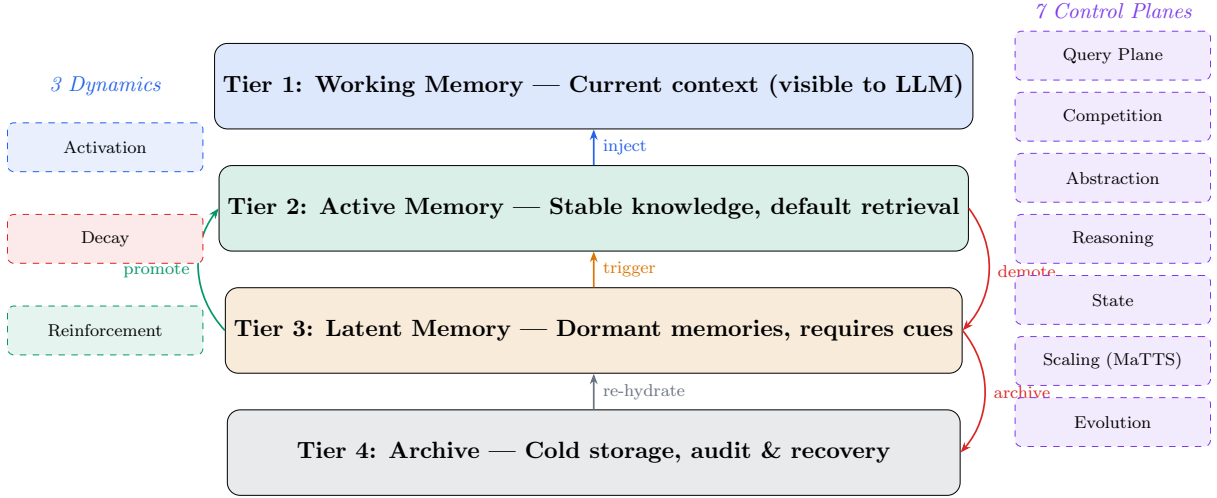


Figure 1: Overall TAM Architecture: 4 storage tiers + 3 dynamics mechanisms + 7 horizontal control planes.

B Summary Comparison Tables

Comparison of 4 Storage Tiers

Table 1: Characteristics comparison of the 4 storage tiers in TAM.

Tier	Speed	Default	Decay	Main Purpose
Working	Very High	—	No	Active context
Active	High	Yes	Slow	Stable knowledge, strategies
Latent	Med	No	Med-Fast	Episodic memory, traces
Archive	Low	No	No	Logging, audit, raw data

Decay Profile by Memory Type

Table 2: Decay rate by memory type — unit %/day.

Memory Type	Decay Rate	Failure-sensitive?	Notes
Episodic	10%	No	Forgets fastest
Semantic	1%	No	Long-term stable
Reasoning	2%	Yes	Penalty upon counter-examples
Style	5%	No	Needs periodic confirmation
System	0.5%	No	Almost never forgets
Abstract	2%	No	Similar to semantic

C Core Algorithms

Algorithm 1: Activation Score — Calculating activation score for memory candidates

Input: Memory record m , query context q , similarity s
Output: Activation score $a \in [0, 1]$

```
1 recency  $\leftarrow \exp(-0.01 \times \text{hours\_since\_last\_use}(m))$ ;  
2 strategy_match  $\leftarrow \text{STRATEGYMATCH}(m.\text{type}, q.\text{intent})$ ;  
3  $a \leftarrow w_1 \cdot s + w_2 \cdot m.\text{importance} + w_3 \cdot \text{recency} + w_4 \cdot m.\text{confidence} + w_5 \cdot \text{strategy\_match}$ ;  
4 return  $\text{clamp}(a, 0, 1)$ ;  
5 Function StrategyMatch( $type, intent$ ):  
6   if  $intent = \text{technical}$  and  $type \in \{\text{reasoning}, \text{system}\}$  then  
7     return 1.0;  
8   else if  $intent = \text{personal}$  and  $type \in \{\text{semantic}, \text{style}\}$  then  
9     return 1.0;  
10  else if  $intent = \text{recall}$  and  $type = \text{episodic}$  then  
11    return 1.0;  
12  else  
13    return 0.5;  
14  end
```

Algorithm 2: Competition — MMR-style diversity selection

Input: Candidate set C , budget k , diversity λ
Output: Winner set W with $|W| \leq k$

```
1  $C \leftarrow \text{DEDUP}(C)$ ;  
2  $W \leftarrow \{\arg \max_{c \in C} \text{score}(c)\}$ ;  
3  $C \leftarrow C \setminus W$ ;  
4 while  $|W| < k$  and  $C \neq \emptyset$  do  
5   foreach  $c \in C$  do  
6      $\text{max\_sim} \leftarrow \max_{w \in W} \text{sim}(c, w)$ ;  
7      $\text{mmr}(c) \leftarrow \lambda \cdot \text{score}(c) - (1 - \lambda) \cdot \text{max\_sim}$ ;  
8   end  
9    $c^* \leftarrow \arg \max_{c \in C} \text{mmr}(c)$ ;  
10   $W \leftarrow W \cup \{c^*\}$ ;  $C \leftarrow C \setminus \{c^*\}$ ;  
11 end  
12 foreach  $c \in C$  (eliminated) do  
13   Mark  $c$  as inhibited by closest winner;  
14 end  
15 return  $W$ ;
```

D Python Source Code Implementation

Directory Structure

```
tiered-agent-memory/  
  tam/  
    __init__.py          # Package init  
    config.py            # Config & hyperparameters  
    models.py            # MemoryRecord, enums  
    pipeline.py          # 12-step orchestrator  
    tiers/  
      working_memory.py  # Tier 1 - Working Memory  
      active_memory.py   # Tier 2 - Active (SQLite + Vector)  
      latent_memory.py   # Tier 3 - Latent (cue-based)  
      archive.py         # Tier 4 - Archive (cold storage)  
    dynamics/  
      activation.py       # Activation scoring  
      decay.py           # Type-aware decay  
      reinforcement.py    # Contrastive signals  
    planes/  
      query_plane.py     # Intent + query rewriting  
      competition.py     # MMR competition  
      abstraction.py      # Abstract memory synthesis  
      reasoning.py        # ReasoningBank  
      state_plane.py     # Conflict + temporal  
      scaling.py          # MaTTS  
      evolution.py       # Self-evolving worker  
    demo.py              # CLI demo  
    tests/
```

Usage Example

Listing 1: TAM Pipeline Initialization and Query

```
from tam import TAMPipeline, TAMConfig, MemoryType  
  
# Initialization  
config = TAMConfig(db_path="my_agent.db")  
pipeline = TAMPipeline(config)  
  
# Adding memory  
pipeline.add_memory(  
    "User likes Python programming",  
    memory_type=MemoryType.SEMANTIC,  
    domain_tags=["technical"],  
)  
  
# Query - runs the 12-step pipeline  
result = pipeline.process("How to fix timeout bug?")  
  
# Result  
print(result["query_context"]["intent"]) # "technical"
```

```
print(result["retrieval"]["winners"])      # Number of winning
      memories
print(result["wm_summary"])                # WM state
```

Technologies Used

Component	Technology
Persistent storage	SQLite (built-in Python)
Vector search	NumPy cosine similarity
Embedding	Hash-based (demo) / sentence-transformers (production)
Competition	MMR — Maximal Marginal Relevance
Test framework	pytest

E References

- [1] C. Packer et al., “MemGPT: Towards LLMs as Operating Systems,” *arXiv:2310.08560*, 2023.
- [2] N. Shinn et al., “Reflexion: Language Agents with Verbal Reinforcement Learning,” *NeurIPS*, 2023.
- [3] S. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” *NeurIPS*, 2023.
- [4] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.
- [5] J. Carbonell and J. Goldstein, “The use of MMR, diversity-based reranking for reordering documents and producing summaries,” *SIGIR*, 1998.
- [6] A. Baddeley, “Working Memory: Theories, Models, and Controversies,” *Annual Review of Psychology*, 2012.
- [7] E. Tulving, “Episodic and semantic memory,” *Organization of Memory*, 1972.
- [8] J. R. Anderson, “A spreading activation theory of memory,” *Journal of Verbal Learning and Verbal Behavior*, 1983.